

חלק א – עליכם לענות על כל השאלות בחלק זה במחברת הבחינה

שאלה 1 (25 נקודות)

נתון מערך חד ממדי `arr` שמלא בספרות (digits) בין 1 ל-9 (כולל), ומספר נוסף שלם חיובי `num`.

אנחנו מעוניינים לבדוק אם אפשר לחבר את כל האיברים במערך כך שסכומם יהיה `num`. אבל, כיוון שבכל התאים במערך יש רק ספרות בודדות, אפשר לשרשר כמה תאים ביחד ולהתייחס למספר שבהם כאילו הם מספרים בני יותר מספרה אחת. מותר וחובה להשתמש בכל תא פעם אחת בלבד. הספרות משורשרות על פי סדרן במערך. אפשר לשרשר רק תאים רצופים!

לדוגמא, אם המספר `num=38` והמערך `arr` הוא:

0	1	2
7	3	1

אז אפשר לבדוק אם $7 + 3 + 1 = 38$ (התשובה היא, כמובן, לא),

אבל צריך לבדוק גם אם $73 + 1 = 38$ (שוב, לא),

אם $7 + 31 = 38$ (הפעם, כן)

ואם $731 = 38$ (לא).

כלומר, יש דרך בה אפשר לחבר את כל האיברים במערך והסכום יהיה המספר `num`.

כתבו שיטה סטטית בוליאנית רקורסיבית המקבלת כפרמטרים מערך חד ממדי `arr` המכיל ספרות בין 1 ל-9 (כולל) ומספר שלם חיובי `num`, ומחזירה `true` אם יש דרך לחבר את כל איברי המערך, כשהם משורשרים למספרים בני ספרה אחת או יותר, כך שסכומם יהיו `num`. אם אין דרך כזו, השיטה תחזיר `false`. כל איבר במערך חייב להשתתף בסכום, ופעם אחת בלבד!

אם יש דרך כזו, השיטה צריכה בנוסף גם להדפיס כפלט את החלוקה של הספרות למספרים כך שסכומם יהיו הפרמטר `num`.

חתימת השיטה :

```
public static boolean digitsSumArray(int[] arr, int num)
```

– דוגמאות נוספות נמצאות בהמשך השאלה –

דוגמאות נוספות:

1. אם איברי המערך הם: {1, 2, 3, 4, 5} ו- $num = 168$
על השיטה להחזיר true ולהציג את הפלט 123 45
כיון ש- $123 + 45 = 168$

2. אם איברי המערך הם: {6, 7} ו- $num = 67$
על השיטה להחזיר true ולהציג את הפלט 67
כיון ש- $67 = 67$

3. אם איברי המערך הם: {8} ו- $num = 8$
על השיטה להחזיר true ולהציג את הפלט 8
כיון ש- $8 = 8$

4. אם איברי המערך הם: {1, 1, 1} ו- $num = 3$
על השיטה להחזיר true ולהציג את הפלט 1 1 1
כיון ש- $1 + 1 + 1 = 3$

לפעמים עשויים להיות כמה פתרונות. למשל:
5. אם איברי המערך הם: {6, 6, 4, 4, 4, 1} ו- $num = 115$
יש כמה פתרונות אפשריים, למשל:

$$6 + 64 + 4 + 41 = 115$$

$$66 + 4 + 44 + 1 = 115$$

$$66 + 44 + 4 + 1 = 115$$

ועוד.

במקרה כזה מספיק להדפיס פתרון אחד, לא משנה איזה מהם. אם למשל נבחר הפתרון האחרון, על השיטה להחזיר true ולהציג את הפלט

$$66 \quad 44 \quad 4 \quad 1$$

6. אם איברי המערך הם: {1, 2} ו- $num = 4$
על השיטה להחזיר false ולא להדפיס דבר, כיון שאין שום דרך להגיע לתוצאה 4 עם הספרות 1 2.

השיטה שתכתבו צריכה להיות רקורסיבית ללא שימוש בלולאות כלל. כך גם כל שיטות העזר שתכתבו (אם תכתבו) לא יכולות להכיל לולאות.

- אפשר להניח שהמערך שהשיטה מקבלת אינו null, ויש בו לפחות תא אחד. כל התאים מכילים ספרות והספרה 0 אינה מופיעה בו. אין צורך לבדוק זאת.
- אפשר להניח שהמספר num הוא שלם חיובי. אין צורך לבדוק זאת.
- אין צורך לדאוג ליעילות השיטה! אבל כמובן שצריך לשים לב לא לעשות קריאות רקורסיביות מיותרות!
- מי שיכתוב שיטה שרק תחזיר true או false ולא תדפיס את הפתרון במקרה והוא קיים, יקבל 15 נקודות לכל היותר (אם הפתרון יהיה נכון).
- אל תשכחו לתעד את מה שכתבתם!

שאלה 2 (25 נקודות)

נתונים שני מערכים חד-ממדיים a ו- b המלאים במספרים שלמים. המערך a ממין בסדר **לא עולה** (כלומר, סדר יורד אבל יכולים להיות מספרים שווים), והמערך b ממין בסדר **לא יורד**.

כתבו שיטה סטטית המקבלת את המערכים a ו- b ומחזירה את האינדקס i הקטן ביותר שמקיים $a[i] == b[i]$, אם קיים כזה. אם לא, השיטה תחזיר -1 .

לדוגמא, אם נתונים המערכים הבאים:

		0	1	2	3	4	5	6	7	8		
a	=	19	19	16	15	15	15	15	13	5		
			↑									
		0	1	2	3	4	5	6	7	8	9	10
b	=	0	12	13	14	14	15	15	19	25	30	35
												↑

אזי השיטה תחזיר את הערך 5 כיוון שהוא הקטן ביותר בו האיבר במערך a שווה לאיבר במערך b . (מסומן באפור).

חתימת השיטה היא:

```
public static int meetingPoint (int[] a, int[] b)
```

שימו לב:

השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.

מה סיבוכיות זמן הריצה וסיבוכיות המקום של השיטה שכתבתם? הסבירו תשובתכם.

אל תשכחו לתעד את מה שכתבתם!

חלק ב - את התשובות לשאלות 3, 4 ו-5 יש לכתוב על גבי השאלון. לא נבדוק תשובות שייכתבו במקום אחר!

שאלה 3 (18 נקודות)

בניח שהמחלקה Node שלהלן מממשת צומת עץ בינרי שערכיו הם מספרים שלמים (int).

```
public class Node
{
    private int _number;
    private Node _leftSon, _rightSon;

    public Node (int number)
    {
        _number = number;
        _leftSon = null;
        _rightSon = null;
    }

    public int  getNumber()      {return _number; }
    public Node getLeftSon()     {return _leftSon; }
    public Node getRightSon()    {return _rightSon; }

    public void setNumber(int number) { _number = number; }
    public void setLeftSon(Node node)  { _leftSon = node; }
    public void setRightSon(Node node) { _rightSon = node; }
}
```

המחלקה BinaryTree מאגדת בתוכה שיטות סטטיות לטיפול **בעץ בינרי**.

בין השיטות במחלקה נתונות השיטות הבאות:

```
public static int what(Node root)
{
    if (root == null)
        return 0;

    return 1 + Math.max(what(root.getLeftSon()),
                        what(root.getRightSon()));
}
```

```

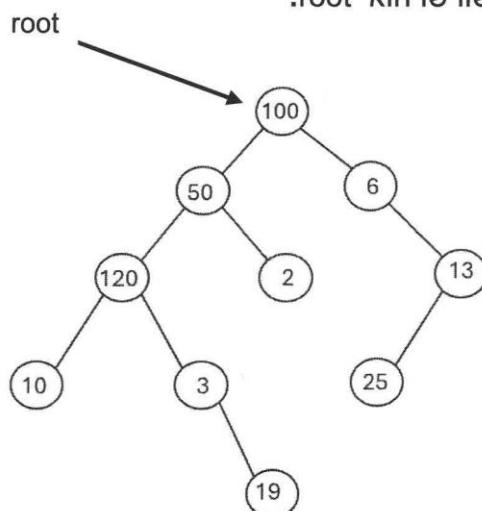
public static void something(Node root, int i, int j)
{
    if (root == null)
        return;
    if (i == j) {
        System.out.print(root.getNumber() + " ");
        return;
    }
    something(root.getLeftSon(), i, j + 1);
    something(root.getRightSon(), i, j + 1);
}

public static void secret(Node root, int i, int limit)
{
    if (i < limit)
    {
        something(root, i, 0);
        System.out.println();
        secret(root, i+1, limit);
    }
}

public static void mystery(Node root)
{
    int x = what(root);
    secret(root, 0, x);
}

```

נתון העץ הבינרי הבא, ששורשו הוא root:



ענו על הסעיפים הבאים:

1. מה יודפס כתוצאה מהקריאה `?System.out.println(what(root));`
התשובה היא: (2 נקודות)

2. מה יודפס כתוצאה מהקריאה `?mystery(root);`
התשובה היא: (6 נקודות)

-	_____
-	_____
-	_____
-	_____
-	_____

3. בשיטה `something` נעשה שינוי. להלן השיטה החדשה (השינויים מודגשים):

```
public static void something(Node root, int i, int j)
{
    if (root == null)
        return;
    if (i == j) {
        System.out.print(root.getNumber() + " ");
        return;
    }
    if (i % 2 == 1)
    {
        something(root.getRightSon(), i, j + 1);
        something(root.getLeftSon(), i, j + 1);
    }
    else
    {
        something(root.getLeftSon(), i, j + 1);
        something(root.getRightSon(), i, j + 1);
    }
}
```


מה יודפס כתוצאה מהקריאה `?mystery(root);` **התשובה היא:** (5 נקודות)

-	_____
-	_____
-	_____
-	_____
-	_____

4. החזרנו את השיטה `something` להיות כפי שהיתה בהתחלה, ושינינו את השיטות `secret` ו-`mystery`. השיטות החדשות הן (השינויים מודגשים):

```
public static void secret(Node root, int i, int limit)
{
    if (i >= limit)
    {
        something(root, i, 0);
        System.out.println();
        secret(root, i-1, limit);
    }
}

public static void mystery(Node root)
{
    int x = what(root);
    secret(root, x, 0);
}
```

מה יודפס כתוצאה מהקריאה `?mystery(root);` **בהנחה שהשיטה `something` חוזרת להיות כפי שהיתה בהתחלה. (ללא השינוי של סעיף 3)** **התשובה היא:** (5 נקודות)

-	_____
-	_____
-	_____
-	_____

שאלה 4 (16 נקודות)

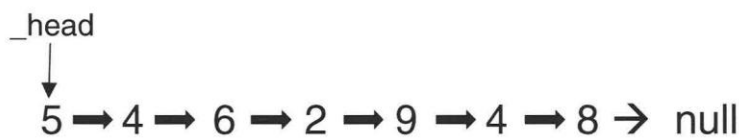
נתונה המחלקה `IntNode` הבאה, המייצגת איבר ברשימה מקושרת חד-סטרית המכילה מספרים שלמים:

```
public class IntNode
{
    private int _value;
    private IntNode _next;

    public IntNode(int val, IntNode n) {
        _value = val;
        _next = n;
    }

    public int getValue()           { return _value; }
    public IntNode getNext()        { return _next; }
}
```

נתונה הרשימה `list` הבאה:



לשם קיצור, נכתוב את הרשימה שלעיל כך, כאשר המספר משמאל הוא ראש הרשימה `_head`
{5, 4, 6, 2, 9, 4, 8}

נתונה רשימה מקושרת חד-סטרית, הממומשת בעזרת המחלקה `IntList` שמכילה את השיטה **what** הכתובה להלן.

הניחו שיש שיטות נוספות במחלקה, כגון שיטה להוספת איבר לרשימה, שיטה למחיקת איבר מהרשימה, וכן השיטה `toString` המחזירה מחרוזת תווים המייצגת את איברי הרשימה.

```
public class IntList
{
    private IntNode _head;

    public IntList()           { _head = null; }
    public IntList(IntNode node) { _head = node; }
```

--- המשך המחלקה בעמוד הבא ---

```

public int what (int x)
{
    int temp = 0;
    IntNode ptr = _head;
    while (ptr != null)
    {
        boolean flag = true;
        if (ptr.getValue() <= x)
        {
            temp++;
            IntNode nextPtr = ptr.getNext();
            while ((nextPtr != null) && flag)
            {
                if (nextPtr.getValue() <= x)
                {
                    temp++;
                    nextPtr = nextPtr.getNext();
                }
                else
                    flag = false;
            }
        }
        ptr = ptr.getNext();
    }
    return temp;
} // end of what

} //end of class IntList

```

ענו על הסעיפים הבאים:

סעיף א (4 נקודות)

נתונה הרשימה list1 הבאה: {5, 1, 2, -1, 14, -2, 1, 10}

נפעיל את השיטה what על הרשימה list1 כאשר הפרמטר לשיטה הוא $x = 3$.

מה יודפס כתוצאה מהקריאה `System.out.println(list1.what(3));` ?

התשובה היא:

– המשך השאלה בעמוד הבא –

סעיף ב (9 נקודות – 3 נקודות לכל תת-סעיף)

נתונה הרשימה list1 הבאה: { 5, 4, 6, 14, 9, 12, 8}

א. כתבו מהו טווח המספרים של x כך שאם נפעיל את השיטה what על הרשימה list1 והפרמטר לשיטה יהיה x , השיטה תחזיר מספר שלם שאינו 0. אם אי אפשר לדעת זאת מהנתונים, או שאין מספר x כזה, נמקו מדוע לא.

התשובה היא:

—

ב. כתבו מהו טווח המספרים של x כך שאם נפעיל את השיטה what על הרשימה list1 והפרמטר לשיטה יהיה x , השיטה תחזיר את הערך 2. אם אי אפשר לדעת זאת מהנתונים, או שאין מספר x כזה, נמקו מדוע לא.

התשובה היא:

ג. נפעיל את השיטה what על הרשימה list1 והפרמטר לשיטה יהיה x כלשהו, כתבו מהו טווח המספרים שיוחזרו מהשיטה. אם אי אפשר לדעת זאת מהנתונים, נמקו מדוע לא.

התשובה היא:

—

סעיף ג (3 נקודות)

מה מבצעת השיטה what באופן כללי, כאשר היא פועלת על רשימה חד-סטריית list המכילה מספרים שלמים, והפרמטר שהיא מקבלת הוא מספר שלם x ? שימו לב, עליכם לתת תיאור ממצה של מה עושה השיטה באופן כללי, ולא תיאור של מה עושה כל שורה בשיטה, או איך היא מבצעת זאת. כלומר, מה המשמעות של הערך המוחזר מהשיטה.

התייחסו למקרי קצה.

התשובה היא:

—

—

—

שאלה 5 (16 נקודות) -

נתונות המחלקות A, B, C ו-D הבאות. כולן באותו פרויקט. כל מחלקה בקובץ נפרד.

```
public abstract class A
{
    protected int _aVal;

    public A() {
        _aVal = 1;
    }
    public A(int aVal) {
        _aVal = aVal;
    }
    public abstract boolean foo(int n);
} // end of class A

//-----

public class B extends A
{
    private int _bVal;

    public B(int aVal, int bVal) {
        super(aVal);
        _bVal = bVal;
    }
} // end of class B

//-----

public class C extends A
{
    private int _cVal;

    public C(int aVal, int cVal) {
        _cVal = cVal;
    }
} // end of class C

//-----
```

```

public class Driver
{
    public static void main (String[] args)
    {
        A b1 = new B(2, 4);
        B b2 = new B(5, 5);
        A c1 = new C(4, 1);
        C c2 = new C(5, 7);

        System.out.println(!b1.foo(2));
        System.out.println( b1.foo(4));
        System.out.println( b2.foo(5));
        System.out.println( c1.foo(1));
        System.out.println(!c2.foo(7));
        System.out.println(!c2.foo(5));
    }
} // end of class Driver

```

סעיף א – (4 נקודות. 2 נקודות לכל תת-סעיף)

כדי שהמחלקות יתקמפלו וירוצו עליכם להוסיף קטעי קוד במחלקות B ו-C. כתבו את הקוד ההכרחי שצריך להוסיף למחלקות B ו-C (בלבד!), על מנת שהתכנית שכתובה בשיטה main במחלקה Driver תתקמפל, תרוץ, וכל ההדפסות בשיטה ידפיסו את הערך true.

שימו לב, אין לשנות את המחלקה A.

עליכם לממש את השיטות תוך שימוש בתכונות המחלקה. בפרט, אינכם יכולים להשתמש במספרים בשיטות. לא בהכרח כל השורות יתמלאו.

1. במחלקה B יש להוסיף את השיטה:

```

public _
{
    _
    _
    _
}

```

2. במחלקה C יש להוסיף את השיטה:

```
public _____  
{  
    _____  
    _____  
    _____  
}
```

סעיף ב - (4 נקודות. 2 נקודות לכל תת-סעיף)

ממשו במחלקות A ו-B את השיטה equals.

שימו לב,

אובייקטים מסוג A יש להשוות לפי התכונה _aVal

אובייקטים מסוג B יש להשוות לפי התכונות _aVal ו- _bVal

גם הפעם יש להשתמש בתכונות המחלקה בלבד ולא במספרים.

1. במחלקה A יש להוסיף את השיטה:

```
public boolean equals (Object obj)  
{  
    if _____  
        return false;  
    return _____  
}
```

2. במחלקה B יש להוסיף את השיטה:

```
public boolean equals (A obj)  
{  
    if _____  
        return false;  
    return super.equals(obj) && _____  
}
```

סעיף ג – (8 נקודות. 2 נקודות לכל תת-סעיף)

ענו על התת-סעיפים הבאים:

1. מה יהיה הערך של המשתנה `aval` עבור אובייקט מסוג **C** שנוצר כך `new C(4,1)` (1 בשיטה `main` במחלקה `driver` ?)

סמנו את התשובה הנכונה:

א. 4

ב. 0

ג. 1

ד. תהיה שגיאת קומפילציה

2. האם השיטה `equals(A obj)` במחלקה `B` דורסת (Overrides) את השיטה `equals(Object obj)` מהמחלקה `A` ?

סמנו את התשובה הנכונה ונמקו:

א. כן, כי

-

ב. לא, כי

-

3. בשיטה `main` במחלקה `driver` נתונה השורה :

`A b1 = new B(2, 4);`

איזו שיטה תתבצע בפועל בקריאה `b1.foo(4)` ?

סמנו את התשובה הנכונה:

א. השיטה `foo` של המחלקה `A`

ב. השיטה `foo` של המחלקה `B`

ג. תהיה שגיאת קומפילציה כי `A` היא אבסטרקטית.

ד. השיטה `foo` של המחלקה `C`

4. בתוך השיטה equals של המחלקה B מופיעה הקריאה super.equals(obj) . למי היא מתייחסת ?

סמנו את התשובה הנכונה:

א. לשיטה equals של המחלקה Object

ב. לשיטה equals של המחלקה A

ג. לבנאי של המחלקה A

ד. היא קוראת לשיטה equals של המחלקה B עצמה.

ב ה צ ל ה !